

Sprint 2 Retrospective

Field	Details
Date	November 7, 2025
Team	LeaveItToUs Development Team
Participants	@Deepthi, @DM, @HB, @SR, @CG

Background

Sprint 2 focused on extending the core functionality of the Employee Leave Management System (ELMS) by implementing HR-level management modules, notification handling, and reporting features. The team also addressed UI refinements postponed from Sprint 1 and strengthened the system's error-handling mechanisms to ensure reliability and usability across modules.

Sprint Duration: November 6 – November 9, 2025

Sprint Goal: Deliver HR policy management, reporting, and system notifications with full integration into the existing authentication and leave workflow.

Deliverables

- ELMS-7: Leave Policy & HR Management
- ELMS-8: Notifications, Reports & Error Handling
- UI refinements carried forward from Sprint 1

Sprint Outcome

- All planned epics were completed successfully within the sprint timeline.
- The sprint burndown chart shows consistent progress, achieving 27 story points completed (0 remaining) before the sprint close.
- Integration between HR management and leave workflows was verified through test cases and team reviews.
- Notification and error-handling modules were finalized with functional testing.

Retrospective Discussion

The team reviewed highlights, challenges, and takeaways from Sprint 2. Below is a summary of the discussion.

What Went Well

- All stories completed with 100% story point closure before sprint end.

- Improved sprint planning and adherence to timelines.
- Successful implementation of automated notifications and reporting modules.
- Mid-sprint reviews helped identify and resolve dependencies early.

What Didn't Go Well

- Some report formatting inconsistencies required last-minute adjustments.
- Unit test coverage for HR module could be expanded.
- Occasional delays in merge approvals due to review backlogs.

Ideas for Improvement

- Allocate dedicated QA testing time before sprint closure.
- Automate report validation to reduce manual checks.
- Introduce pre-merge testing scripts to speed up code reviews.

Start / Stop / Keep Doing Table

Start Doing	Stop Doing	Keep Doing
Writing automated test scripts for major modules	Delaying merge reviews until end of sprint	Maintaining structured sprint goals and timelines
Conducting code review sessions twice per sprint	Relying solely on manual validation for reports	Effective collaboration between Dev and QA teams
Using CI/CD pipeline for deployment testing	Handling small UI fixes outside of planned stories	Consistent mid-sprint check-ins and progress tracking

Action Items

1. Enhance test automation — increase unit test coverage, especially for HR and reporting modules.
2. Streamline merge reviews — assign reviewers in advance to prevent end-sprint delays.
3. Implement report templates — standardize output formats to minimize UI inconsistencies.
4. Integrate CI/CD validation — automate build checks and test runs before merges.
5. Continue mid-sprint checkpoints — maintain effective monitoring and dependency tracking.
6. Plan sprint review documentation — summarize key outcomes and demo notes for reference.

Team Feedback Summary

Team performance in Sprint 2 demonstrated clear improvement from Sprint 1. The sprint was completed on schedule with full story point closure, reflecting better planning, estimation, and coordination. Mid-sprint reviews helped the team maintain focus and deliver all modules within scope. The team agreed to prioritize automation, testing efficiency, and consistent code reviews in future sprints. Overall morale was positive, and the team's collaboration and velocity improved significantly.